

## ✓ INTELIGENCIA ARTIFICIAL

### Laboratorio 2: Búsqueda con Información (A\* y heurísticas)

#### Indicaciones generales:

- Las respuestas deben contar con **fundamento teórico**.
- Cualquier indicio de plagio resultará en la anulación de la prueba.
- Debe presentar sus respuestas en base a los resultados de ejecución en los casos que se solicite. **No se calificarán aquellas respuestas que no presenten un resultado de ejecución o que no concuerden con este.**
- Subir el cuadernillo con el nombre **Lab02\_código.ipynb**, donde código es su código PUCP de 8 dígitos.
- Está prohibido el uso de herramientas de IA. En caso se detecte el uso de estas herramientas la calificación será 00.

**Usted deberá completar el código en las secciones indicadas con "COMPLETAR"**

## ✓ Búsqueda en SpaceWorld

Un arqueólogo espacial ha recibido una misteriosa señal proveniente de un rincón remoto del universo. Ha conseguido encontrar su origen, pero necesita tu ayuda para que el piloto automático de su nave encuentre la mejor ruta para llegar hasta allí sin estrellarse contra el Sol.

El objetivo es llegar al origen de la señal (el objetivo 'E'). Además, el espacio tiene elementos, como planetas y satélites, que el arqueólogo le gustaría visitar en su camino ya que representa cierto "valor" al conocimiento científico. Sin embargo, su principal preocupación es el combustible de la nave, pues es limitado. Además, existen obstáculos, como asteroides, que requieren más combustible (costo) para atravesarlos, y el Sol, que es completamente impasable.

#### Resumen de los elementos del espacio:

- 'S' (Sol): Impasable, costo infinito, valor 0.
- 'E' (Objetivo): Costo 1, valor 100.
- 'P' (Planeta): Costo -0.5 (puede recargar combustible en el planeta, por eso el costo es negativo), valor 15.
- 'T' (Satélite): Costo -0.5 (puede recargar combustible en el satélite), valor 50.
- 'A' (Asteroide): Costo 5, valor 0.
- ' ' (Espacio vacío): Costo 1, valor 0.

```
#Carga de librerías
import numpy as np
import random
import math
from collections import deque
import heapq
import plotly.graph_objects as go
from plotly.offline import iplot
import plotly.express as px
# Semilla
random.seed(42)
np.random.seed(42)
```

#### > Clases base - No modifique este código, solo ejecute la celda

[Mostrar código](#)

#### > Visualización - No modifique este código, solo ejecute la celda

[Mostrar código](#)

## ✓ Clase SpaceWorld

Implementa el mundo SpaceWorld

```
#No modificar
class SpaceWorld:
    def __init__(self, n=(25, 25, 25)):
```

```

self.dimX, self.dimY, self.dimZ = n
self.object_colors = {}
self.object_sizes = {}
self.grid = self.generate_solar_system(n)
self.start = self.find_planet_position('P1')
self.grid[self.start[0]][self.start[1]][self.start[2]] = 'P'
self.goal = self.find_planet_position('E')

def find_planet_position(self, symbol):
    for i in range(self.dimX):
        for j in range(self.dimY):
            for k in range(self.dimZ):
                if self.grid[i][j][k] == symbol:
                    return (i, j, k)
    return None

def isPassable(self, x, y, z):
    return self.grid[x][y][z] not in BLOCK_DEADLY

def isBlocked(self, x, y, z):
    return not self.isPassable(x, y, z)

def getBlock(self, x, y, z):
    return self.grid[x][y][z]

def getX(self): return self.dimX
def getY(self): return self.dimY
def getZ(self): return self.dimZ
def getStartCell(self): return self.start
def getGoalCell(self): return self.goal

def generate_solar_system(self, n, seed=42):
    """Función que genera el sistema solar en base a una semilla. No modificar"""
    if seed is not None:
        random.seed(seed)
        np.random.seed(seed)

    x, y, z = n
    space = np.full((x, y, z), ' ', dtype=object)
    center_x, center_y, center_z = x//2, y//2, z//2

    space[center_x, center_y, center_z] = 'S'
    self.object_sizes[(center_x, center_y, center_z)] = SPACE_CONFIG['SUN']['size']

    eye_x, eye_y, eye_z = 0, 0, 0
    space[eye_x, eye_y, eye_z] = 'E'
    self.object_sizes[(eye_x, eye_y, eye_z)] = SPACE_CONFIG['EYE']['size']

    start_x, start_y, start_z = x-1, y-1, z-1
    space[start_x, start_y, start_z] = 'P1'
    self.object_sizes[(start_x, start_y, start_z)] = random.randint(*SPACE_CONFIG['PLANET']['size_range'])

    num_planets = random.randint(35, 45)
    planets_placed = []
    for _ in range(num_planets):
        attempts = 0
        while attempts < 50:
            px = random.randint(2, x-3)
            py = random.randint(2, y-3)
            pz = random.randint(2, z-3)

            dist_from_sun = math.sqrt((px-center_x)**2 + (py-center_y)**2 + (pz-center_z)**2)
            dist_from_eye = math.sqrt((px-eye_x)**2 + (py-eye_y)**2 + (pz-eye_z)**2)

            if dist_from_sun > 3 and dist_from_eye > 1 and space[px, py, pz] == ' ':
                space[px, py, pz] = 'P'
                size = random.randint(*SPACE_CONFIG['PLANET']['size_range'])
                self.object_sizes[(px, py, pz)] = size
                planets_placed.append((px, py, pz))
                break
            attempts += 1

    num_satellites = random.randint(14, 16)
    for _ in range(num_satellites):
        if planets_placed:
            parent_planet = random.choice(planets_placed)
            px, py, pz = parent_planet
            for dx in [-1, 0, 1]:
                for dy in [-1, 0, 1]:
                    for dz in [-1, 0, 1]:
                        if dx == 0 and dy == 0 and dz == 0:

```

```

        continue
    sx, sy, sz = px + dx, py + dy, pz + dz
    if (0 <= sx < x and 0 <= sy < y and 0 <= sz < z and
        space[sx, sy, sz] == ' '):
        space[sx, sy, sz] = 'T'
        size = random.randint(*SPACE_CONFIG['SATELLITE']['size_range'])
        self.object_sizes[(sx, sy, sz)] = size
        break
    else:
        continue
    break
else:
    continue
break

num_asteroids = random.randint(120, 160)
for _ in range(num_asteroids):
    attempts = 0
    while attempts < 30:
        ax = random.randint(0, x-1)
        ay = random.randint(0, y-1)
        az = random.randint(0, z-1)

        if space[ax, ay, az] == ' ':
            space[ax, ay, az] = 'A'
            size = random.randint(*SPACE_CONFIG['ASTEROID']['size_range'])
            self.object_sizes[(ax, ay, az)] = size
            break
        attempts += 1

    return space

def __str__(self):
    return f"SpaceWorld({self.dimX}x{self.dimY}x{self.dimZ}) - Objetivo en {self.goal}"

```

```

#Complete donde se indique COMPLETAR
class ExplorationProblem(SearchProblem):
    def __init__(self, space):
        """Constructor que recibe el espacio"""
        self.space = space
        self.initial = space.getStartCell()
        self.goal = space.getGoalCell()
        self.numNodesExpanded = 0
        self.expandedNodeSet = {}

    def __isValidState(self, state):
        """Retorna True si el estado es una celda válida y no mortal"""
        x, y, z = state

        #####
        # COMPLETAR - Verificar que todas coordenadas estén dentro de los límites del espacio
        if x < 0 or x >= self.space.getX():
            return False
        if y < 0 or y >= self.space.getY():
            return False
        if z < 0 or z >= self.space.getZ():
            return False
        #####
        return not self.space.isBlocked(x, y, z)

    def actions(self, state):
        """Retorna acciones legales: movimiento en 6 direcciones (3D)"""
        x, y, z = state
        actions = []

        directions = [
            (1, 0, 0, 'D'), # Abajo/X+
            (-1, 0, 0, 'U'), # Arriba/X-
            (0, 1, 0, 'R'), # Derecha/Y+
            (0, -1, 0, 'L'), # Izquierda/Y-
            (0, 0, 1, 'F'), # Adelante/Z+
            (0, 0, -1, 'B') # Atrás/Z-
        ]

        #####
        # COMPLETAR - Agregar a actions las direcciones válidas (de las 6 posibles). Tip: Usar self.__isValidState()
        for dx, dy, dz, accion in directions:
            print(dx, dy, dz, accion);
            new_state = (x + dx, y + dy, z + dz)
            print(new_state);
            if(self.__isValidState(new_state)):
                actions.append(accion)
        print(actions)

```

```
#####
return actions

def result(self, state, action):
    """Retorna el estado resultante de ejecutar la acción"""
    x, y, z = state

    #####
    # COMPLETAR - Calcular las nuevas coordenadas según la acción
    dx, dy, dz = 0, 0, 0

    # Mapear acción a cambio de coordenadas
    if action == 'D': dx = 1
    if action == 'U': dx = -1
    if action == 'R': dy = 1
    if action == 'L': dy = -1
    if action == 'F': dz = 1
    if action == 'B': dz = -1

    new_state = (x + dx, y + dy, z + dz)
    #####

    return new_state

def goal_test(self, state):
    """Retorna True si el estado es el objetivo"""
    return state == self.goal

def path_cost(self, c, state1, action, state2):
    """Retorna el costo del camino, solo considera consumo de combustible (costo)"""
    x, y, z = state2
    block_type = self.space.getBlock(x, y, z)
    c += COST_BLOCKS.get(block_type, 1)
    return c
```

## > Algoritmos de búsqueda

Se implementan algoritmos clásicos de búsqueda informada y no informada:

- **graph\_search:** Algoritmo general de búsqueda con memoria de estados visitados. La frontera puede ser una cola FIFO (BFS) o una pila (DFS).
- **best\_first\_graph\_search:** Expande el nodo con menor valor de una función de evaluación  $f$  (por ejemplo, heurística).
- **astar\_search:** Caso especial de búsqueda best-first donde  $f = \text{path\_cost} + \text{heuristic}$ , es decir, búsqueda A\*.

Estos algoritmos permiten encontrar caminos óptimos en el entorno tridimensional.

↳ 1 celda oculta

## ✓ Heurísticas espaciales para A\*

Las heurísticas implementadas a continuación permiten estimar la distancia restante al objetivo para guiar el algoritmo A\*:

- **Distancia Euclidiana:** Calcula la distancia en línea recta (como vuela una nave) entre el estado actual y el objetivo.

$$h_{euclidiana}(n) = \sqrt{(x_{goal} - x_n)^2 + (y_{goal} - y_n)^2 + (z_{goal} - z_n)^2}$$

- **Distancia Manhattan:** Suma las diferencias absolutas en cada dimensión (movimiento ortogonal en 3D).

$$h_{manhattan}(n) = |x_{goal} - x_n| + |y_{goal} - y_n| + |z_{goal} - z_n|$$

- **Heurística nula:** Siempre retorna 0, haciendo que A\* se comporte como búsqueda de costo uniforme.

$$h_{nula}(n) = 0$$

Estas heurísticas ayudan a priorizar los nodos más prometedores durante la búsqueda.

```
# DEBE COMPLETAR EL CÓDIGO DE CADA HEURÍSTICA

def euclidean_dist(node, problem):
    """Distancia en línea recta desde la posición actual hasta el objetivo"""
    x1, y1, z1 = node.state
    #####
    # COMPLETAR - Calcular la distancia euclidiana entre el estado actual y el objetivo
    #####
    return math.sqrt( math.pow(problem.goal[0] - x1, 2) + math.pow(problem.goal[1] - y1, 2) + math.pow(problem.goal[2] - z1, 2) )

def manhattan_dist(node, problem):
    """Distancia Manhattan (suma de diferencias absolutas)"""
```

```

x1, y1, z1 = node.state
#####
# COMPLETAR - Calcular la distancia Manhattan entre el estado actual y el objetivo
return abs( problem.goal[0] - x1) + abs(problem.goal[1] - y1) + abs(problem.goal[2] - z1)

#####

def null_heuristic(node, problem):
    """Heurística nula (A* se convierte en búsqueda de costo uniforme)"""
    #####
    # COMPLETAR - Implementar la heurística nula
    return 0
    #####

```

```

mundo = SpaceWorld((15, 15, 15))
problem = ExplorationProblem(mundo)

heuristics = [
    ("Euclidiana", euclidean_dist),
    ("Manhattan", manhattan_dist),
    ("Nula", null_heuristic),
]

```

## ▼ Probar las heurísticas

Ejecute el algoritmo A\* con cada una de las heurísticas implementadas

```

# No hace falta modificar nada aquí
for nombre, heur in heuristics:
    print(f"\nProbando heurística: {nombre}")
    nodo_solucion, nodos_visitados = astar_search(problem, heur)
    if nodo_solucion:
        ganancia = 0
        for node in nodo_solucion.path():
            block = mundo.getBlock(*node.state)
            ganancia += VALUE_BLOCKS.get(block, 0)
        print(f"Solución encontrada con costo {nodo_solucion.path_cost}")
        print(f"Ganancia acumulada: {ganancia}")
        print(f"Nodos expandidos: {len(nodos_visitados)}")
        print(f"Número de acciones: {len(nodo_solucion.solution())}")
        print("Ruta de acciones:", nodo_solucion.solution())
        simulate_system_3d(mundo, nodo_solucion.path())
    else:
        print("No se encontró solución.")

```



Probando heurística: Euclidiana

```

1 0 0 D
(15, 14, 14)
-1 0 0 U
(13, 14, 14)
0 1 0 R
(14, 15, 14)
0 -1 0 L
(14, 13, 14)
0 0 1 F
(14, 14, 15)
0 0 -1 B
(14, 14, 13)
['U', 'L', 'B']
1 0 0 D
(14, 14, 14)
-1 0 0 U
(12, 14, 14)
0 1 0 R
(13, 15, 14)
0 -1 0 L
(13, 13, 14)
0 0 1 F
(13, 14, 15)
0 0 -1 B
(13, 14, 13)
['D', 'U', 'L', 'B']
1 0 0 D
(15, 13, 14)
-1 0 0 U
(13, 13, 14)
0 1 0 R
(14, 14, 14)
0 -1 0 L
(14, 12, 14)
0 0 1 F
(14, 13, 15)
0 0 -1 B
(14, 13, 13)
['U', 'R', 'L', 'B']
1 0 0 D
(15, 14, 13)
-1 0 0 U
(13, 14, 13)
0 1 0 R
(14, 15, 13)
0 -1 0 L
(14, 13, 13)
0 0 1 F
(14, 14, 14)
0 0 -1 B
(14, 14, 12)
['U', 'L', 'F', 'B']
1 0 0 D
(14, 13, 14)
-1 0 0 U
(12, 13, 14)
0 1 0 R
(13, 14, 14)
0 -1 0 L
(13, 12, 14)
0 0 1 F
(13, 13, 15)
0 0 -1 B
(13, 13, 13)
['D', 'U', 'R', 'L', 'B']
1 0 0 D
(14, 14, 13)
-1 0 0 U
(12, 14, 13)
0 1 0 R
(13, 15, 13)
0 -1 0 L
(13, 13, 13)
0 0 1 F
(13, 14, 14)
0 0 -1 B
(13, 14, 12)
['D', 'U', 'L', 'F', 'B']
1 0 0 D
(15, 13, 13)
-1 0 0 U
(13, 13, 13)
0 1 0 R
(14, 14, 13)
0 -1 0 L
(14, 12, 13)
0 0 1 F
(14, 13, 14)

```